

Replication: Targeting for Long-Term Outcomes

Yang, Eckles, Dhillon, and Aral (2024), Management Science 70(6), on the authors' synthetic data

Weiheng Zhang

2026-09-05

Contents

1	The paper and the question	1
2	The synthetic data and what they allow	2
2.1	The design	3
3	Step 1, which we cannot run: the surrogate index	3
4	Step 2: doubly robust scores with <code>dr_scores()</code>	3
5	Step 3: policy optimization with <code>policy_learn()</code>	5
6	Step 4: off-policy evaluation with <code>policy_value()</code>	5
7	Beyond the paper: validating the CATE signal	7
8	Beyond the paper: what the policy looks at	8
9	Beyond the paper: bootstrap Thompson sampling	9
10	What the workflow teaches	10
11	References	10

1 The paper and the question

Yang, Eckles, Dhillon, and Aral (2024) want a rule that assigns a discount to digital subscribers of *The Boston Globe* so as to maximize long-term (18-month or 3-year) revenue. The difficulty is that the long-term outcome is not observed when the rule has to be chosen. Their answer has three steps.

1. **Surrogate index.** On historical data where the long-term outcome is observed, fit $\tilde{Y} = E_H[Y \mid S, X]$ with gradient boosting, where S are surrogates observed in the first one to six months after treatment (revenue and articles read by section) and X are pre-treatment covariates. Impute $\tilde{Y}$ for every subscriber in the experiment. Under ignorability, surrogacy, and comparability, the value of any policy on $\tilde{Y}$ equals its value on Y (their Propositions 1 and 2); for the *optimal* policy, only sign preservation of the conditional average treatment effect is needed (Proposition 3).
2. **Doubly robust scores and policy optimization.** Cross-fit outcome models $\hat{\mu}(x, a)$ for $\tilde{Y}$, use the

known design propensities $\pi_D(a | x)$, form the scores of their equation (19),

$$\hat{\gamma}_a(X_i) = \hat{\mu}(X_i, a) + \frac{\mathbb{1}\{A_i = a\}}{\pi_D(a | X_i)} (\tilde{Y}_i - \hat{\mu}(X_i, a)),$$

and solve the cost-sensitive classification of equation (21) with a gradient-boosting classifier.

3. **Off-policy evaluation.** Value of the learned rule on a held-out test sample by the doubly robust estimator of equation (17), against the status quo of treating no one, with percentile intervals from 1,000 bootstrap draws.

The doubly robust score is the same object as the pseudo-outcome of Section 08 of the lecture notes, and every step below is one function of the package applied to it. Their published results: the policy treats 21 percent of subscribers and is worth about \$40 per subscriber over three years (95 percent interval [\$10, \$75]); policies learned on surrogates from months 2 to 6 beat the status quo, policies learned on short-term revenue alone do not, and the six-month surrogate policy is statistically indistinguishable from the policy learned on realized 18-month revenue (their Figure 3).

2 The synthetic data and what they allow

The real data are under a non-disclosure agreement. The authors provide a synthetic file with 1,000 rows sampled from Experiment 1 and regenerated with `synthpop` (their notebook 5), revenues rescaled by an undisclosed constant. The treatment and design propensity columns are kept from the original rows; every other column is synthesized. The file contains the outputs of the authors' steps 1 and 2 that need data we do not have: the surrogate index of 18-month revenue built from the first $m = 1, \dots, 6$ months of surrogates (`rev_m`), and their cross-fitted outcome models (`y0_m`, `y1_m`). What it does not contain: the realized 18-month revenue, short-term revenue, Experiment 2, and the historical data. So the replication runs steps 2 and 3, exactly what the authors' notebook 6 does on the same file, and their notebook's figure is the number to match. Table 1, Figure 3(a)'s true-revenue reference, and Figures 3(c) and 3(d) cannot be produced from these data.

```
dat <- read.csv(file.path(data_path, "synthetic.csv"))
x_all <- setdiff(names(dat), c("subscriber_id", "treated", "p_treated",
                             grep("^rev_|^y0_|^y1_", names(dat), value = TRUE)))
x_cov <- x_all[sapply(dat[x_all], function(v) length(unique(v)) > 1)]
c(n = nrow(dat), treated = sum(dat$treated), covariates = length(x_cov),
  dropped_constant = length(x_all) - length(x_cov))
#>           n          treated      covariates dropped_constant
#>        1000             16           57              4
```

Table 1: Synthetic Experiment 1 data: 1,000 subscribers, 16 treated. Revenues are rescaled by an undisclosed constant.

Variable	Mean	SD	Min	Max
Design propensity <code>p_treated</code>	0.0207	0.0417	0.00588	0.458
Risk score (predicted churn)	0.000999	0.00134	0.0000110	0.00945
Articles read, last 6 months	260.	434.	0	4008.
Surrogate index of 18-month revenue, 1-month surrogates	140.	17.1	8.62	167.
Surrogate index of 18-month revenue, 2-month surrogates	138.	26.3	10.0	160.
Surrogate index of 18-month revenue, 3-month surrogates	138.	28.6	10.3	160.
Surrogate index of 18-month revenue, 4-month surrogates	139.	30.6	9.70	160.
Surrogate index of 18-month revenue, 5-month surrogates	139.	33.1	8.08	160.
Surrogate index of 18-month revenue, 6-month surrogates	139.	34.6	8.07	160.

Only 16 subscribers are treated, with design propensities between 0.006 and 0.46. Every treated observation therefore carries an inverse-propensity weight of the order of 36, and every value estimate below is noisy. That is the synthetic file's nature, not a choice.

2.1 The design

In the real experiment the design policy assigned subscribers with higher predicted churn to treatment with higher probability, keeping $0 < \pi_D < 1$ for everyone. In the synthetic file the propensity is the original one but the risk score is synthesized, so the link between them is broken (correlation -0.03). Figure 1 shows both facts: propensities spread over two orders of magnitude, and no relation to the synthesized risk score.

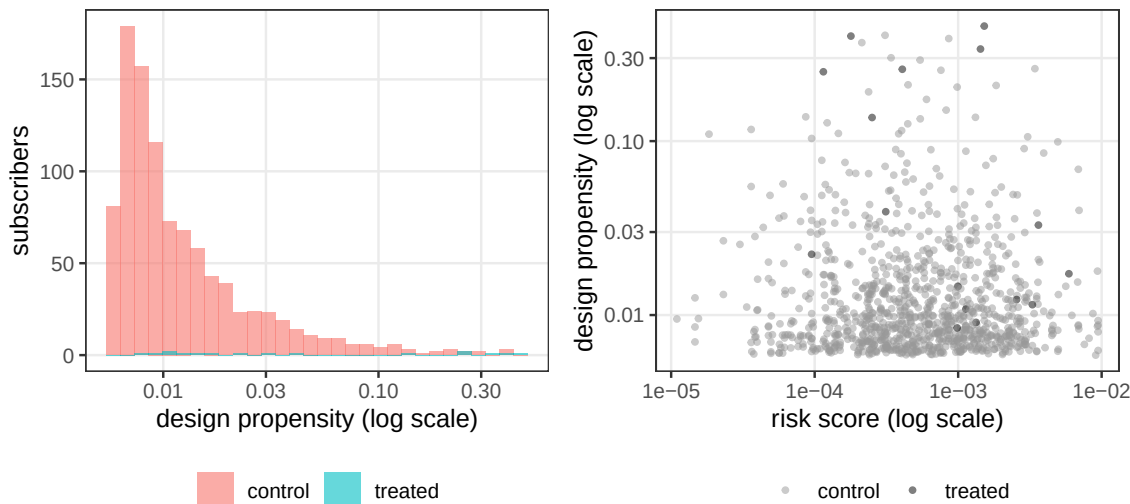


Figure 1: Design propensity of treatment by treatment status (left) and against the synthesized risk score (right), log scales.

3 Step 1, which we cannot run: the surrogate index

The authors regress 3-year (or 18-month) revenue on covariates and on the first m months of surrogates in historical data from 2015 to 2018 with `xgboost` (depth 2, 150 rounds), then predict for the experiment's subscribers. The synthetic file supplies the prediction, `rev_m`, for each horizon. Two things about this step matter for reading the results.

- Comparability (their Assumption 3) requires the conditional distribution of long-term revenue given surrogates to be the same in the historical and the experimental data. It is untestable here and the paper's own validation (Figure 3(a), 3(d)) is the evidence.
- Because the same surrogate index is the outcome at every step, a policy learned on `rev_m` is evaluated on `rev_m`. The paper's real validation evaluates on realized revenue instead; on the synthetic file that validation is out of reach.

4 Step 2: doubly robust scores with `dr_scores()`

Their equation (19) with two arms and the known design propensity is the AIPW pseudo-outcome of `dr_scores()` with the nuisances supplied and no clipping. We build one scores object per horizon and check the formula against the authors' notebook line by line.

```
make_scores <- function(m, rows = seq_len(nrow(dat))) {
  d <- dat[rows, ]
  d$y <- d[[paste0("rev_", m)]]
  d$mu0 <- d[[paste0("y0_", m)]]
  d$mu1 <- d[[paste0("y1_", m)]]
  dr_scores(d, y = "y", d = "treated", x = x_cov,
            p_hat = "p_treated", mu0_hat = "mu0", mu1_hat = "mu1", p_clip = c(0, 1))
}
sc <- lapply(1:6, make_scores)
```

```

# the authors' construction (notebook 6): y0_dr, y1_dr, tau_dr
authors_tau <- function(m) {
  with(dat, {
    y <- dat[[paste0("rev_", m)]]; y0 <- dat[[paste0("y0_", m)]]; y1 <- dat[[paste0("y1_", m)]]
    y0_dr <- ifelse(treated == 1, y0, y0 + (y - y0) / (1 - p_treated))
    y1_dr <- ifelse(treated == 0, y1, y1 + (y - y1) / p_treated)
    y1_dr - y0_dr
  })
}
max(sapply(1:6, function(m) max(abs(sc[[m]]$score - authors_tau(m)))))
#> [1] 4.547474e-13

```

The scores agree to machine precision. Their mean is the doubly robust ATE; the ATT uses the same nuisances with the score of Section 04. The paper's Figure 3(a) plots the ATT on the surrogate index by horizon against the ATT on realized revenue; Figure 2 reproduces the layout without the reference line.

```

effects <- bind_rows(lapply(1:6, function(m) {
  d <- sc[[m]]$data
  att <- est_aipw(d, "y", "treated", p_hat = "p_treated", mu0_hat = "mu0", mu1_hat = "mu1",
    estimand = "ATT", p_clip = c(1e-6, 1 - 1e-6))
  tibble(month = m,
    ATE = sc[[m]]$ate$estimate, ATE_se = sc[[m]]$ate$std.error,
    ATT = att$estimate, ATT_se = att$std.error)
}))
effects %>% mutate(across(-month, ~ round(.x, 2)))
#> # A tibble: 6 x 5
#>   month   ATE ATE_se   ATT ATT_se
#>   <int> <dbl> <dbl> <dbl> <dbl>
#> 1     1  2.13  1.15  4.76  3.11
#> 2     2  7.84  1.77  9.52  3.62
#> 3     3  5.1   1.28  9.33  3.93
#> 4     4 11.8   3.88 11.4   4.7
#> 5     5  9.97  3.76 10.4   4.14
#> 6     6  7.3   3.07 12.5   4.01

```

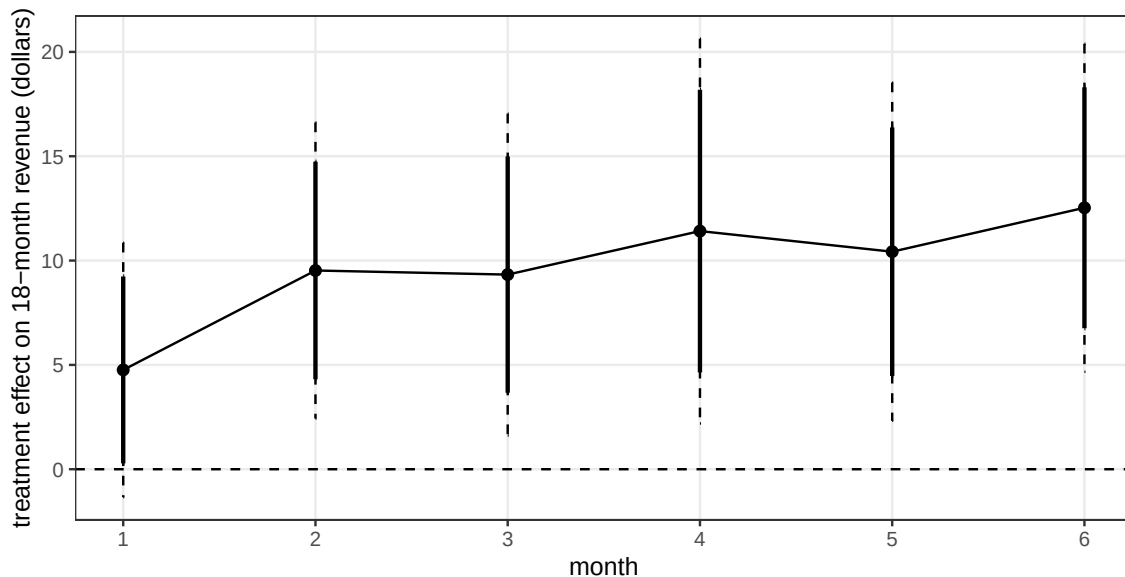


Figure 2: Treatment effects on the surrogate index of 18-month revenue by the number of months of surrogates used, in the layout of the paper's Figure 3(a): ATT (points), 75 percent (solid) and 95 percent (dashed) normal intervals from the influence function. The paper's horizontal reference lines need realized revenue, which the synthetic file does not contain.

Two features of these estimates carry over to everything below. The point estimates grow with the horizon,

as in the paper, where the effect of an eight-week discount is only visible once the surrogates cover the months after it ends. And the standard errors grow with the horizon too, because the outcome models fit the later surrogate indices less well (R2 of mu0: 0.39, 0.30, 0.32, 0.25, 0.25, 0.24).

5 Step 3: policy optimization with `policy_learn()`

The authors split the file 80/20 with seed 101 (`train_id <- sort(sample(n, 0.8 n))`), and fit `mlr`'s cost-sensitive weighted-pairs wrapper around an `xgboost` classifier with pre-tuned hyperparameters, using the misclassification costs (`max(tau, 0)`, `max(-tau, 0)`) for the two actions. With two actions that wrapper is weighted classification with label $\mathbb{1}\{\hat{\tau}_i > 0\}$ and weight $|\hat{\tau}_i|$, which is what `policy_learn(method = "classifier")` fits. We use the same split and hyperparameters, and add two policy classes the paper does not consider: the depth-two policy tree of Athey and Wager (2021), and the budgeted rule that treats the top share of a DR-learner CATE at the share the boosting policy chooses.

```
set.seed(101)
train_id <- sort(sample(nrow(dat), nrow(dat) * 0.8))
test_id <- setdiff(seq_len(nrow(dat)), train_id)
sc_train <- lapply(1:6, make_scores, rows = train_id)
sc_test <- lapply(1:6, make_scores, rows = test_id)
xgb_learner <- lrn("classif.xgboost", predict_type = "prob", eval_metric = "logloss",
  eta = 0.3, max_depth = 6, gamma = 0, colsample_bytree = 0.6,
  min_child_weight = 1, subsample = 1, nrounds = 150, nthread = 1)
rf_final <- lrn("regr.ranger", num.trees = 500, min.node.size = 20)

# Not cached: knitr's chunk cache drops the task backend of mlr3 learners,
# after which they can no longer predict. Fitting takes about a minute.
fit_policies <- function(m) {
  set.seed(101)
  xgb <- policy_learn(sc_train[[m]], x = x_cov, method = "classifier", learner = xgb_learner, holdout = 0)
  tree <- policy_learn(sc_train[[m]], x = x_cov, method = "tree", depth = 2, engine = "policytree", holdout = 0)
  cate <- cate_learner(scores = sc_train[[m]], x_het = x_cov, method = "dr", learner_final = rf_final)
  share <- max(mean(predict(xgb, sc_test[[m]]$data)), 0.05)
  budget <- policy_learn(sc_train[[m]], method = "budget", budget = share, tau_hat = cate, holdout = 0)
  list(xgboost = xgb, tree = tree, budget = budget, cate = cate)
}
pols <- lapply(1:6, fit_policies)
```

The boosting policy treats between 20 and 53 percent of the test subscribers depending on the horizon (the paper: 21 percent on the real data with 3-year revenue). The training value of every class is optimistic, the boosting classifier's most of all: with 150 rounds of depth six it fits the training scores almost perfectly. The test values are the ones to read.

6 Step 4: off-policy evaluation with `policy_value()`

Their equation (17) evaluated for a deterministic rule against the rule that treats no one is the difference of two means of scores, so it is the mean of $\pi(X_i)\hat{\gamma}_i$ with $\hat{\gamma}_i$ the doubly robust CATE score; that is what `policy_value()` returns, with a standard error from the influence function. The authors use 1,000 bootstrap draws of the test rows and report 75 and 95 percent percentile intervals; `n_boot` adds them.

```
eval_tab <- bind_rows(lapply(1:6, function(m) {
  v <- policy_value(sc_test[[m]], pols[[m]]$xgboost, n_boot = params$n_boot, seed = 101)
  B <- attr(v, "boot")[, 1]
  tibble(month = m, estimate = v$estimate, se_if = v$std.error,
    if_low = v$conf.low, if_high = v$conf.high,
    boot_se = v$boot.se, boot_95_low = v$boot.low, boot_95_high = v$boot.high,
    boot_75_low = quantile(B, 0.125), boot_75_high = quantile(B, 0.875))
}))
```

Table 2: Policies optimized on the surrogate index built from the first m months, by policy class. Values are dollars per subscriber over treating no one, doubly robust, on the 800 training and 200 test rows.

Month	Policy class	Share treated (train)	Share treated (test)	Value, train	Value, test	SE, test
1	Gradient boosting (paper)	0.53	0.52	5.62	1.29	0.89
	Policy tree, depth 2	0.58	0.59	3.80	0.53	0.68
	Top share by DR-learner CATE	0.52	0.49	4.90	1.26	0.90
2	Gradient boosting (paper)	0.54	0.53	10.30	4.31	1.65
	Policy tree, depth 2	0.75	0.73	8.07	4.20	1.66
	Top share by DR-learner CATE	0.53	0.61	9.21	10.21	6.35
3	Gradient boosting (paper)	0.41	0.35	10.06	1.80	1.49
	Policy tree, depth 2	0.86	0.86	6.55	4.64	3.10
	Top share by DR-learner CATE	0.35	0.42	8.95	3.17	1.66
4	Gradient boosting (paper)	0.34	0.29	19.68	2.73	1.78
	Policy tree, depth 2	0.65	0.64	14.66	7.83	7.79
	Top share by DR-learner CATE	0.30	0.29	17.53	10.47	7.74
5	Gradient boosting (paper)	0.36	0.26	17.82	4.52	1.81
	Policy tree, depth 2	0.71	0.62	12.08	1.74	2.06
	Top share by DR-learner CATE	0.26	0.25	15.92	8.72	7.57
6	Gradient boosting (paper)	0.32	0.20	16.68	3.40	1.62
	Policy tree, depth 2	0.81	0.81	9.71	0.04	2.41
	Top share by DR-learner CATE	0.21	0.18	14.80	8.43	6.70

Table 3: Value of the boosting policy over treating no one on the 200 test rows: influence-function (IF) intervals from policy_value() and percentile intervals from 1,000 bootstrap draws of the test rows, the authors' construction.

Month	Value	SE (IF)	95% low (IF)	95% high (IF)	SE (boot)	95% low (boot)	95% high (boot)
1	1.29	0.89	-0.46	3.05	0.88	-0.15	3.23
2	4.31	1.65	1.08	7.53	1.68	1.29	8.12
3	1.80	1.49	-1.12	4.71	1.53	-0.87	5.10
4	2.73	1.78	-0.75	6.21	1.83	-0.58	6.56
5	4.52	1.81	0.96	8.07	1.84	1.11	8.35
6	3.40	1.62	0.23	6.57	1.63	0.48	6.92

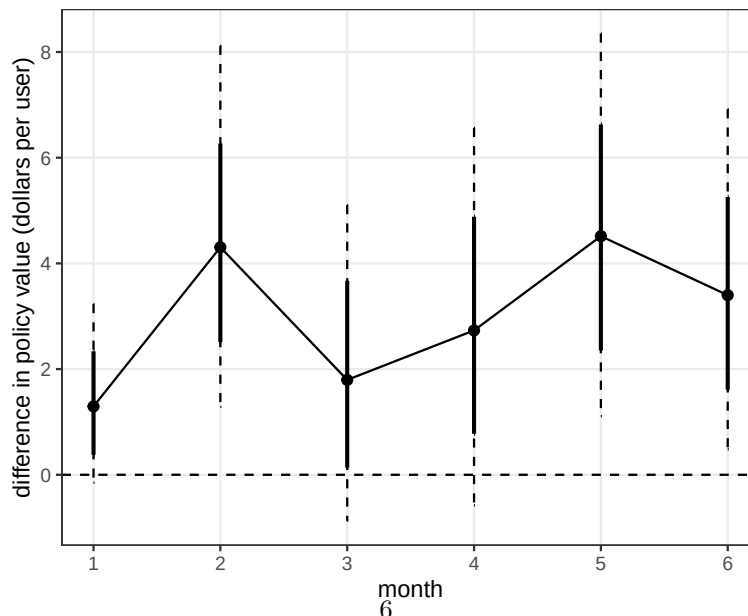


Figure 3: Value of the policy optimized on the surrogate index built from the first m months, relative to treating no one, in the layout of the paper's Figure 3(b) and of the authors' notebook 6: solid bars are 75 percent and dashed bars 95 percent bootstrap percentile intervals (1,000 draws of the test rows).

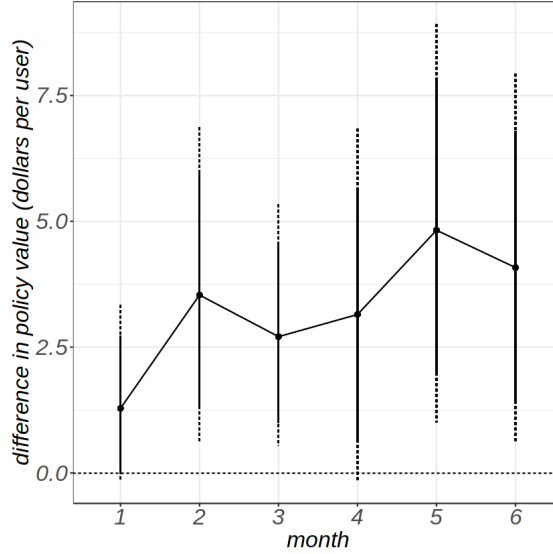


Figure 4: The same figure as produced by the authors’ notebook 6 on the same file with the same split (image embedded in their notebook).

The two figures agree in shape and scale. The authors’ values are about 1.3, 3.5, 2.7, 3.2, 4.8, and 4.1 dollars per user for months 1 to 6; ours are 1.3, 4.3, 1.8, 2.7, 4.5, 3.4. Both start near \$1 at one month and rise to \$4 to \$5 by month 5; ours dip at months 3 and 4 where theirs do not. Their 95 percent intervals exclude zero from month 2 on; ours do at months 2, 5, 6. The differences come from the gradient-boosting fit, which is not identical across `xgboost` versions and draws its column subsamples at random, not from the scores, which are identical. One slip in the authors’ notebook 6 is corrected here: their month-3 policy is optimized on `rev_6` with the month-3 outcome models, a copy-paste error absent from their notebook 3; we use `rev_3`.

Influence-function and bootstrap standard errors agree to within 3 percent, as they should for a sample mean. The bootstrap costs 1,000 predictions per policy and buys nothing here; it is the right tool when the statistic is not a mean, for instance when the policy itself is refitted in each draw (Section 9).

The paper reports values against treating no one because that was the status quo. The other natural benchmark is treating everyone:

```
bind_rows(lapply(c(2, 6), function(m) {
  cbind(month = m, policy_value(sc_test[[m]], pols[[m]]$xgboost, baseline = "all"), c("share_treated", "estimate", "std.error"))
}))
#>   month share_treated estimate std.error
#> 1     2             0.530 -5.685025  6.155842
#> 2     6             0.205 -3.382273  6.838803
```

Against treating everyone the learned rule loses in point estimate, by an amount within one standard error of zero. The reason is Section 4: the average effect of the discount on the surrogate index is positive at every horizon, so treating everyone earns the ATE per subscriber, and a rule that treats a fifth to a half of them captures less than that in these data. The paper’s status quo was treating no one, so its values do not answer this question; the real data, with an ATE that is a few dollars against a discount cost the surrogate index nets out, may well differ. A firm should report both benchmarks.

7 Beyond the paper: validating the CATE signal

The policy’s value says that the rule’s sign is informative. The diagnostics of Section 08 ask more of the CATE model behind it. On the test scores at six months we validate the DR-learner from the budgeted rule (a random forest on the scores). With about three treated subscribers among the 200 test rows, the honest

expectation is wide intervals.

```
val <- cate_validate(sc_test[[6]], pols[[6]]$cate, n_groups = 4, n_boot = 499, seed = 1)
val$blp
#>
#> term estimate std.error statistic p.value conf.low
#> ATE 6.7811058 7.0111356 0.9671908 0.3334487 -6.9604674
#> tau_c tau_hat (centered) 0.5866514 0.4287121 1.3684042 0.1711856 -0.2536088
#> conf.high
#> 20.522679
#> tau_c 1.426912
val$areas
#> statistic estimate std.error conf.low p.value
#> 1 AUTOC 10.347996 7.042999 -1.2367059 0.07088115
#> 2 AUQC 3.943896 2.485646 -0.1446275 0.05629387
```

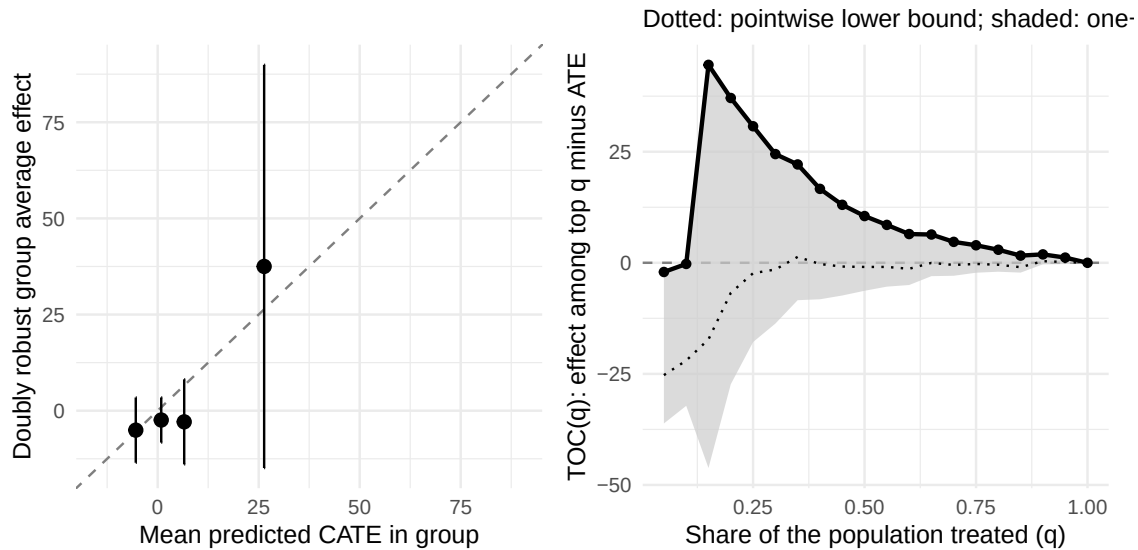


Figure 5: Validation of the DR-learner CATE on the test rows (six-month surrogate index). Left: doubly robust group average effect by quartile of predicted CATE against the mean prediction. Right: TOC curve with pointwise (dotted) and one-sided simultaneous (shaded) 95 percent lower bounds.

The curves are wide and the heterogeneity slope is imprecise, as expected with 16 treated subscribers in the file. The exercise still shows what a targeting paper can report next to a policy value: whether the ranking that the policy uses is calibrated, and at what share of the population the targeting gain is concentrated.

8 Beyond the paper: what the policy looks at

The paper's Online Appendix D.4 reports variable importance and partial dependence of the boosting policy on the real data (top variables: risk score, tenure, sports articles in the last six months; non-monotone relations). The fitted classifier is available as `$model`, so the same tools apply.

```
imp <- xgboost::xgb.importance(model = pols[[6]]$xgboost$model$model)
head(imp, 10)
#>
#> Feature Gain Cover Frequency
#> <char> <num> <num> <num>
#> 1: risk_score 0.12000698 0.22752023 0.18969264
#> 2: last_6month 0.11943238 0.06723231 0.06799131
#> 3: metro_last_6month 0.06090366 0.04767359 0.05246818
#> 4: sports_last_6month 0.05719201 0.02480679 0.03601366
#> 5: sunday_magazine_last_6month 0.05644867 0.04397482 0.01241850
#> 6: metro_last_3month 0.04605968 0.04107210 0.03942875
#> 7: news_last_6month 0.04417646 0.03844779 0.03725551
#> 8: metro_last_month 0.03361150 0.02920407 0.02763117
```

```
#> 9:  member_center_last_6month 0.03032082 0.02218603 0.01707544
#> 10:  business_last_6month 0.03020786 0.03867027 0.03477181
```

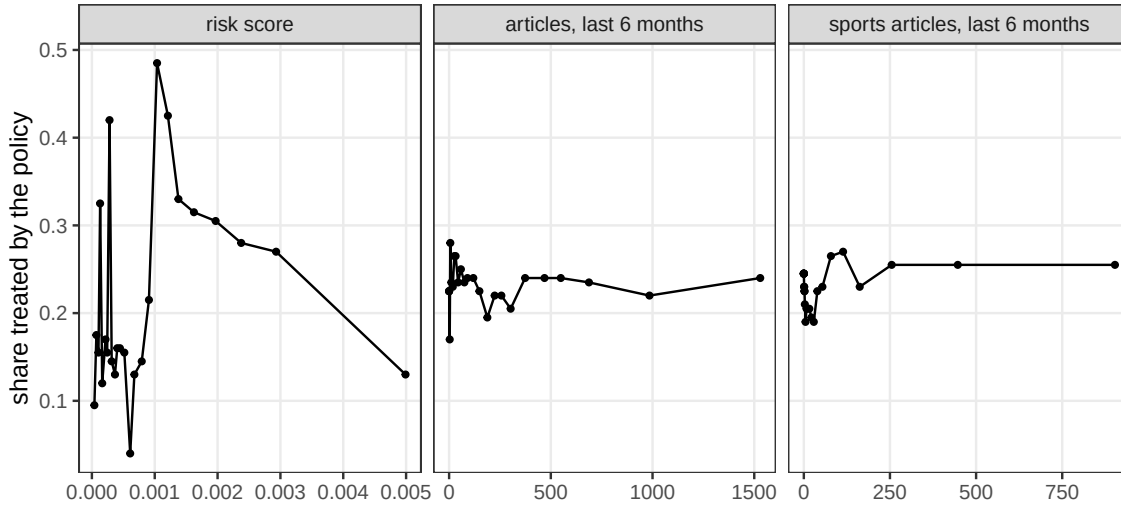


Figure 6: Share of subscribers the six-month boosting policy treats as one covariate is moved over its range with the others held at their observed values (partial dependence on the test rows), for the three variables closest to the paper’s Appendix D.4: risk score, total articles in the last six months (the file has no tenure), and sports articles in the last six months.

A depth-two policy tree is the transparent alternative. It is fitted to the same scores (Table 2 gives its value) and can be read directly:

```
pols[[6]]$tree$model
#> policy_tree object
#> Tree depth: 2
#> Actions: 1 2
#> Variable splits:
#> (1) split_variable: opinion_last_6month split_value: 6
#> (2) split_variable: metro_last_month split_value: 2
#> (4) * action: 2
#> (5) * action: 1
#> (3) split_variable: metro_last_month split_value: 107
#> (6) * action: 2
#> (7) * action: 1
```

9 Beyond the paper: bootstrap Thompson sampling

Section 4.2.3 of the paper implements the learned policy in Experiment 2 with exploration: a subscriber is treated with probability equal to the share of bootstrap replicates in which the refitted policy treats them (their equation 23). We refit the boosting classifier on 200 resamples of the training rows and read off the exploration probabilities on the test rows.

```
bts <- replicate(params$n_bts, {
  idx <- sample(train_id, length(train_id), replace = TRUE)
  sc_b <- make_scores(6, rows = idx)
  pol_b <- policy_learn(sc_b, x = x_cov, method = "classifier", learner = xgb_learner, holdout = 0)
  predict(pol_b, test_data)
})
pi_bts <- rowMeans(bts)
c(mean_treat_prob = mean(pi_bts), share_interior = mean(pi_bts > 0.1 & pi_bts < 0.9),
  agreement_with_point_policy = mean((pi_bts > 0.5) == (predict(pols[[6]]$xgboost, test_data) == 1)))
#> mean_treat_prob share_interior
#> 0.2278 0.5650
```

```
#> agreement_with_point_policy
#> 0.8600
```

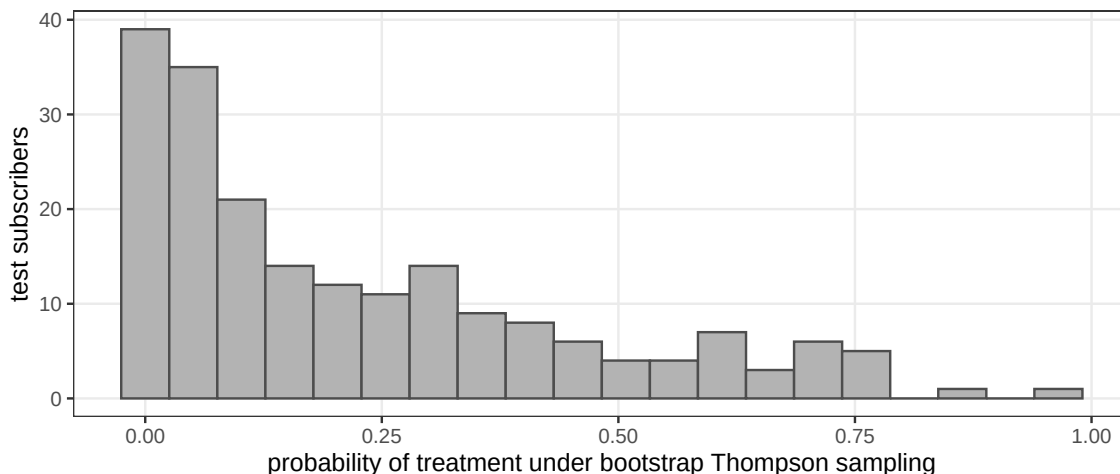


Figure 7: Bootstrap Thompson sampling probabilities of treatment on the test rows: the share of refitted policies that treat each subscriber.

56 percent of the test subscribers have a treatment probability strictly between 0.1 and 0.9: with 16 treated units the policy is uncertain for more than half of the population, and the exploration this induces is what allows a second experiment to improve on it.

10 What the workflow teaches

- **One object.** The doubly robust score built once in Step 2 is the ATE and ATT (its mean), the label of the policy classifier (its sign and magnitude), the value of any rule (its mean over the treated set), the outcome of every validation statistic, and the reward of the bootstrap Thompson sampling loop. `dr_scores()` reproduces the authors' scores to machine precision; every later number is a different average of the same vector.
- **Where surrogacy enters.** Only in the outcome column. The policy machinery never sees the long-term outcome; the surrogate index stands in for it, and the assumptions of Section 4.1 of the paper are what make the substitution innocent. The paper's real contribution is the validation on realized revenue, which the synthetic file cannot reproduce.
- **Where the uncertainty comes from.** Sixteen treated subscribers with propensities as low as 0.006. The doubly robust score divides by those propensities, and every interval in this document is wide for that reason. The paper's real data have about 45,000 subscribers and 806 treated; the width of its intervals is the same phenomenon scaled by $\sqrt{n}$.
- **Bootstrap or influence function.** For a policy value the two agree, and the influence function is free. The bootstrap earns its cost when the policy is refitted inside it.

11 References

- Athey, Susan, Raj Chetty, Guido W. Imbens, and Hyunseung Kang. 2019. "The Surrogate Index: Combining Short-Term Proxies to Estimate Long-Term Treatment Effects More Rapidly and Precisely." NBER Working Paper 26463.
- Athey, Susan, and Stefan Wager. 2021. "Policy Learning with Observational Data." *Econometrica*.
- Dudík, Miroslav, Dumitru Erhan, John Langford, and Lihong Li. 2014. "Doubly Robust Policy Evaluation and Optimization." *Statistical Science*.

- Eckles, Dean, and Maurits Kaptein. 2014. “Thompson Sampling with the Online Bootstrap.” arXiv:1410.4009.
- Yang, Jeremy, Dean Eckles, Paramveer Dhillon, and Sinan Aral. 2024. “Targeting for Long-Term Outcomes.” *Management Science* 70(6), 3841-3855.
- Zhou, Zhengyuan, Susan Athey, and Stefan Wager. 2023. “Offline Multi-Action Policy Learning: Generalization and Optimization.” *Operations Research*.